NOVA IMS Information Management School

2024/2025

DEEP LEARNING

RARE SPECIES CLASSIFICATION

MASTER IN DATA SCIENCE AND ADVANCED ANALYTICS

GROUP 13

- AFONSO LOPES, 20211540
- ANA ISABEL DUARTE, 20240545
- INÊS ARÚJO, 20240532
- LEONOR MIRA, 20240658
- RITA MATOS, 20211642

Table of Contents

1. Introduction 2
2. Exploratory Data Analysis and Data Preprocessing 2
   2.1. Class Imbalance 2
   2.2. Resizing Strategy 2
   2.3. Data Split 3
   2.4. Data Augmentation 3
3. Model Development 4
   3.1. Evaluation Strategy and Optimization Considerations 4
   3.2. Designing and Evaluating Custom CNN Architectures 4
   3.3. Transfer Learning with Pre-Trained Networks 5
   3.4. Refinements and Fine-Tuning of Pre-Trained Models 6
   3.5. Final Model Selection and Performance Analysis 6
4. Conclusion 7
5. References 8
6. Appendix 9

1. Introduction

Deep learning has become a powerful tool for solving complex image classification tasks, including biological applications where traditional methods often fall short. Recent projects like iNaturalist [1] and LifeCLEF [2] have shown how image-based models can assist in large-scale species identification, supporting research and conservation efforts.

Building on this progress, our project focuses on classifying rare species from images, using a dataset from the *BioCLIP: A Vision Foundation Model for the Tree of Life* [3] study. The aim is to predict the family of a species based solely on its image.

Accurate and scalable species classification has broad applications in biodiversity monitoring, ecological research, and environmental protection. By developing a model capable of this task, we aim to contribute to automated solutions that support the identification and preservation of rare species.

2. Exploratory Data Analysis and Data Preprocessing

The preprocessing phase focused on analyzing image properties and preparing the data for model training through standardization and resizing, ensuring compatibility with the architecture and supporting later augmentation steps.

2.1. Class Imbalance

Given the objective of this project, the initial phase involved analyzing the class distribution of our target variable. We computed basic descriptive statistics to assess potential imbalance and understand the spread across the 202 categories. The number of images per class ranged from as few as 29 to as many as 300, with most classes falling between 30 and 60 images (Figure 1). These findings point to a considerable imbalance in class representation, which could bias the model toward more frequent categories [4]. A similar pattern was observed at the phylum level, where the majority of images belonged to a single phylum, further emphasizing the need to account for imbalance (Figure 2). This asymmetry motivated the use of data augmentation techniques to mitigate its effects and improve the model's generalization across all categories.

2.2. Resizing Strategy

To inspect the visual and technical characteristics of the images, we confirmed that all images were in RGB format with pixel intensity values ranging from 0 to 255. We also noticed significant variation in image resolutions, with some being much larger than others (Figure 3). Since deep learning models require consistent input dimensions, this disparity called for resizing all images to a uniform resolution to ensure compatibility and efficient processing during model training.

To prepare the dataset for training, we explored different image resizing strategies, as deep learning models require consistent input dimensions. Our goal was to strike a balance between computational efficiency and the preservation of visual detail - critical for a task involving complex biological features. Multiple resized versions of the dataset were created to support experimentation with different image sizes during model training. While smaller image

dimensions reduce memory usage and significantly speed up training, excessively downscaling the images risks losing important visual cues that are essential for accurate classification.

We experimented with both square and rectangular resizing approaches. Square images are commonly used in computer vision models and often lead to better performance due to architectural expectations in many pre-trained convolutional neural networks. However, our original dataset contained predominantly rectangular images, forcing them into a square format introduced noticeable distortions - compressing and tightening the visual content in a way that could hinder the model's ability to learn meaningful features. Rectangular resizing, on the other hand, preserved the original aspect ratio, maintaining the visual integrity of the images. Both strategies were applied across dedicated train, validation, and test splits, and their outputs were visually inspected to assess quality. This preprocessing step was critical in guiding our decision on which resizing method to adopt in the modeling phase.

2.3. Data Split

Regarding the split of our data, cross-validation was considered as a potential evaluation strategy but was rejected due to its high computational cost and the added difficulty of implementation. Each fold would require training a full model, which is highly time-consuming given the dataset size and model architecture. Instead, we opted for a hold-out approach, which, while less robust than cross-validation, was more feasible in this context and still provided reliable performance. The data was split into train/validation/test, where 80% of the data was allocated to the training set, and the remaining 20% was divided equally into validation and test sets (10% each). This approach allowed for efficient evaluation while ensuring that the model was tested on unseen data.

Furthermore, to prepare the labels for model training, we applied one-hot encoding to the target variable. This transformation converted each family into a binary vector format, compatible with the loss function defined later in the modeling phase.

2.4. Data Augmentation

To enhance the model's performance and generalization capabilities, we explored different parameter combinations for data augmentation [5]. Two augmentation strategies were tested: the first applied more aggressive transformations, which introduced substantial visual distortions, while the second used more moderate values aimed at maintaining image integrity. After evaluation, the second augmentation method not only preserved more relevant visual features but also consistently led to better results during training and validation.

This second, more balanced augmentation strategy was ultimately adopted as the standard across all models developed in the project - both the custom convolutional architectures and the models built on top of pre-trained networks. It served as a robust foundation for improving generalization while maintaining visual coherence in the input data.

Both augmentation pipelines included rescaling, normalizing pixel values from the [0, 255] range to [0, 1], and a series of transformations such as random rotations, width and height shifts, shearing, zooming, and horizontal flips. Additionally, missing pixels were handled using a nearest-

pixel fill mode. The second approach also introduced slight adjustments in brightness, which helped improve the model's ability to handle varying lighting conditions.

3. Model Development

3.1. Evaluation Strategy and Optimization Considerations

Accuracy is one of the most commonly used metrics for classification problems due to its simplicity and ease of interpretation. However, as identified in our Exploratory Data Analysis, there is significant class imbalance within the provided dataset. The presence of many underrepresented species families meant that accuracy alone would not provide an adequate evaluation of our model's performance.

Given the class imbalance, the F1 score was chosen as the primary metric, as it combines both precision and recall. Unlike accuracy, the F1 score is more suitable for imbalanced datasets as it considers both false positives and false negatives, providing a more balanced evaluation of the model's performance, especially for the minority classes [6].

For the loss function, we used categorical cross-entropy, which is the standard loss function for multiclass classification tasks when the target labels are one-hot encoded. This loss function compares the predicted class probabilities to the one-hot encoded true labels and helps the model adjust its weights accordingly during training.

In addition to focusing on performance metrics, we also prioritized the efficiency and speed of our model. Given the large number of images in the dataset and the limited computational resources, optimizing the model for efficient processing was crucial. The time constraints imposed by the project deadline further emphasized the need for a model that could be trained and evaluated within the available timeframe, without compromising the accuracy or F1 score.

3.2. Designing and Evaluating Custom CNN Architectures

We developed and evaluated nine custom models using the Keras library, all based on a straightforward Convolutional Neural Network (CNN) architecture. The main objective was to explore how changes in image resolution, batch size, and architecture complexity influenced the model's ability to generalize on a large-scale, multi-class classification problem (Table 1).

Each model shared a common structure: an input layer tailored to the image resolution, followed by two convolutional layers with ReLU activation and 'same' padding, interleaved with max-pooling layers. These were followed by a flattening layer, a dense layer with 128 units, and finally a softmax-activated output layer with 202 neurons - matching the number of classes in our dataset. All models were trained using the Adam optimizer.

To manage overfitting and training stability, every model was trained with a set of callbacks: early stopping [7], learning rate reduction on plateau [8], and model checkpointing [9]. These dynamic training strategies helped control training time while ensuring the model converged to the best possible weights based on validation performance.

Among the nine custom models, Model 6 offered the best trade-off between performance and generalization. Despite not having the highest raw F1 score, it demonstrated a low train-

validation gap and more stable loss curves, which suggests better robustness. Model 5 achieved the highest validation F1 (0.1912), but its noticeably higher training F1 and lower loss pointed to overfitting, making it less suitable for general use. In an effort to achieve better results, we built on top of model 6, attempting to design a deeper and more complex version of the architecture to push performance further. However, it proved to be extremely computationally expensive, and, consequently, unfeasible. We also experimented with a custom data augmentation pipeline that introduced more aggressive transformations, including color manipulation, but these changes did not lead to better results.

Although none of the custom models showed particularly strong final results, this stage was crucial in establishing a performance baseline. It also allowed us to identify key architectural and hyperparameter patterns, which guided further experimentation with transfer learning and more complex models.

3.3. Transfer Learning with Pre-Trained Networks

After testing custom CNN architectures, we explored transfer learning using three widely recognized base models: VGG16 [10], EfficientNetB0 [11], and DenseNet201 [12], all sourced from the Keras applications module. These models come pre-loaded with weights trained on ImageNet, allowing us to repurpose their learning for our 202-class classification task. In addition to that, all images were standardized to a square format of 224x224 pixels, aligning with the input requirements of widely adopted convolutional neural network architectures. Although this resolution introduced some degree of distortion to non-square images, it enabled full compatibility with established pre-trained models and leveraged their transfer learning capabilities (Table 2).

When training the pre-trained models, we applied a similar augmentation strategy as we did with the custom CNNs, with slight adjustments to better suit the pre-trained architectures. Since these models are trained on ImageNet with specific normalization schemes, we omitted the rescale step applied in our augmentation and instead used the corresponding preprocess_input function from the Keras library to format images appropriately. This ensured that the input data was properly aligned with the original training conditions of each base model.

For all configurations, we followed a consistent fine-tuning strategy. The base model was initially frozen to preserve its pre-trained weights during early training. On top, we added a lightweight classification head consisting of global average pooling, a dense layer with 256 units (ReLU activation + L2 regularization), batch normalization, dropout (0.5), and a final softmax layer for output. We used the same training setup as in the custom models.

Among the three, EfficientNetB0 clearly stood out. It achieved the highest validation F1 score of 0.7146 with strong overall generalization (train F1: 0.7799) and relatively low loss. This model trained for 41 epochs and showed high performance and no clear signs of overfitting (Figures 4, 5 and 6).

Overall, transfer learning proved to be a clear upgrade over the custom CNNs in terms of performance. For the next stage of experimentation, we selected EfficientNetB0 as our baseline and built on top of it to push for even better results.

3.4. Refinements and Fine-Tuning of Pre-Trained Models

A series of targeted improvements were implemented throughout the pre-trained model experiments to address the complexities inherent to the rare species classification task. Each modification was introduced with the objective of optimizing model performance, enhancing generalization, and mitigating the impact of class imbalance.

Efforts included optimizing data augmentation pipelines, adjusting fine-tuning strategies by selectively unfreezing layers, and systematically varying hyperparameters such as learning rate, batch size, and optimizer configuration. Regularization techniques, including dropout and early stopping, were incorporated to minimize overfitting. In addition, class weighting and alternative loss functions were explored to counter the pronounced class imbalance and better evaluate model performance, particularly for underrepresented classes.

While our initial model was trained only to predict the family, its confusion matrix showed widespread errors, with misclassifications scattered across many different families (Figure 7). To address this, we developed a second model that incorporated both family and phylum information during training [13]. This modification aimed to leverage hierarchical relationships between classes to guide the model toward more structured predictions.

When comparing the confusion matrices of the two models (focusing on the 20 most confused classes), we observed a clear difference: the family-only model produced more diffuse errors, while the model trained with phylum labels showed a much cleaner pattern, with confusion concentrated between fewer classes (Figure 8).

The overall performance of the model with phylum information was similar but slightly worse than the best family-only model. However, it produced more structured and consistent mistakes, which could be valuable for biologically meaningful applications.

In addition, we also experimented with EfficientNetB6, which utilizes larger input images compared to the base EfficientNetB0 (528x528). Given the hypothesis that using larger images might improve performance by providing more detailed visual information, we tested this model to evaluate whether it would enhance accuracy, particularly for complex species classification tasks.

3.5. Final Model Selection and Performance Analysis

Despite the range of enhancements introduced throughout the project, the EfficientNetB0-based architecture originally implemented (Pre-trained Model 2.0 analyzed previously) remained the top performer. While the model demonstrated strong overall performance, with good generalization to the test set, the results suggest there is still room for improvement (Table 3). The accuracy and F1 score indicate that the model is able to correctly classify a significant portion of the species, but a meaningful number of misclassifications still occur, particularly for the more challenging or underrepresented species. This points to the potential for further improvements, which could push the model closer to optimal performance in the rare species classification task.

4. Conclusion

In conclusion, multiple challenges had to be addressed during this project, primarily due to class imbalance in the dataset and limited computational resources. All experiments were conducted locally in VS Code, which, for example, made it prohibitive to develop more complex architectures or use cross-validation, forcing us to rely on a simple hold-out validation strategy. As a result, in an effort to use a more targeted and computationally mindful approach, we opted to make one change at a time, observing its impact, and iterating only when improvements were confirmed.

For future work, a key focus should be on the most confused classes identified in the confusion matrices. While our initial model predicted only families and showed scattered errors, a second model trained with both family and phylum labels produced more structured confusion, even if slightly less accurate overall. Building on this, future improvements could include targeted data augmentation for problematic classes and curriculum learning to gradually refine classification granularity [14]. Access to more powerful hardware would also enable deeper networks, larger input sizes, and robust cross-validation.

5. References

[1] "iNaturalist Computer Vision Explorations · iNaturalist." Accessed: Apr. 25, 2025. [Online]. Available: https://www.inaturalist.org/pages/computer_vision_demo

[2] A. Joly *et al.*, "Overview of LifeCLEF 2022: An Evaluation of Machine-Learning Based Species Identification and Species Distribution Prediction," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 13390 LNCS, pp. 257–285, 2022, doi: 10.1007/978-3-031-13643-6_19.

[3] S. Stevens *et al.*, "BIOCLIP: A Vision Foundation Model for the Tree of Life".

[4] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, Oct. 2018, doi: 10.1016/J.NEUNET.2018.07.011.

[5] "A Complete Guide to Data Augmentation | DataCamp." Accessed: Apr. 29, 2025. [Online]. Available: https://www.datacamp.com/tutorial/complete-guide-data-augmentation?dc_referrer=https%3A%2F%2Fwww.google.com%2F

[6] "F1 Score." Accessed: Apr. 29, 2025. [Online]. Available: https://graphite-note.com/understanding-the-importance-of-f1-score-in-machine-learning-3/

[7] "EarlyStopping." Accessed: Apr. 29, 2025. [Online]. Available: https://keras.io/api/callbacks/early_stopping/

[8] "ReduceLROnPlateau." Accessed: Apr. 29, 2025. [Online]. Available: https://keras.io/api/callbacks/reduce_lr_on_plateau/

[9] "ModelCheckpoint." Accessed: Apr. 29, 2025. [Online]. Available: https://keras.io/api/callbacks/model_checkpoint/

[10] "VGG16 and VGG19." Accessed: Apr. 29, 2025. [Online]. Available: https://keras.io/api/applications/vgg/

[11] "EfficientNet B0 to B7." Accessed: Apr. 29, 2025. [Online]. Available: https://keras.io/api/applications/efficientnet/

[12] "DenseNet." Accessed: Apr. 29, 2025. [Online]. Available: https://keras.io/api/applications/densenet/

[13] "An Overview of Multi-Task Learning in Deep Neural Networks." Accessed: Apr. 29, 2025. [Online]. Available: https://www.researchgate.net/publication/317649808_An_Overview_of_Multi-Task_Learning_in_Deep_Neural_Networks

[14] "Curriculum learning. The concept of curriculum learning in… | by Noha Nekamiche | AIGuys | Medium." Accessed: Apr. 29, 2025. [Online]. Available: https://medium.com/aiguys/curriculum-learning-83b1b2221f33

6. Appendix

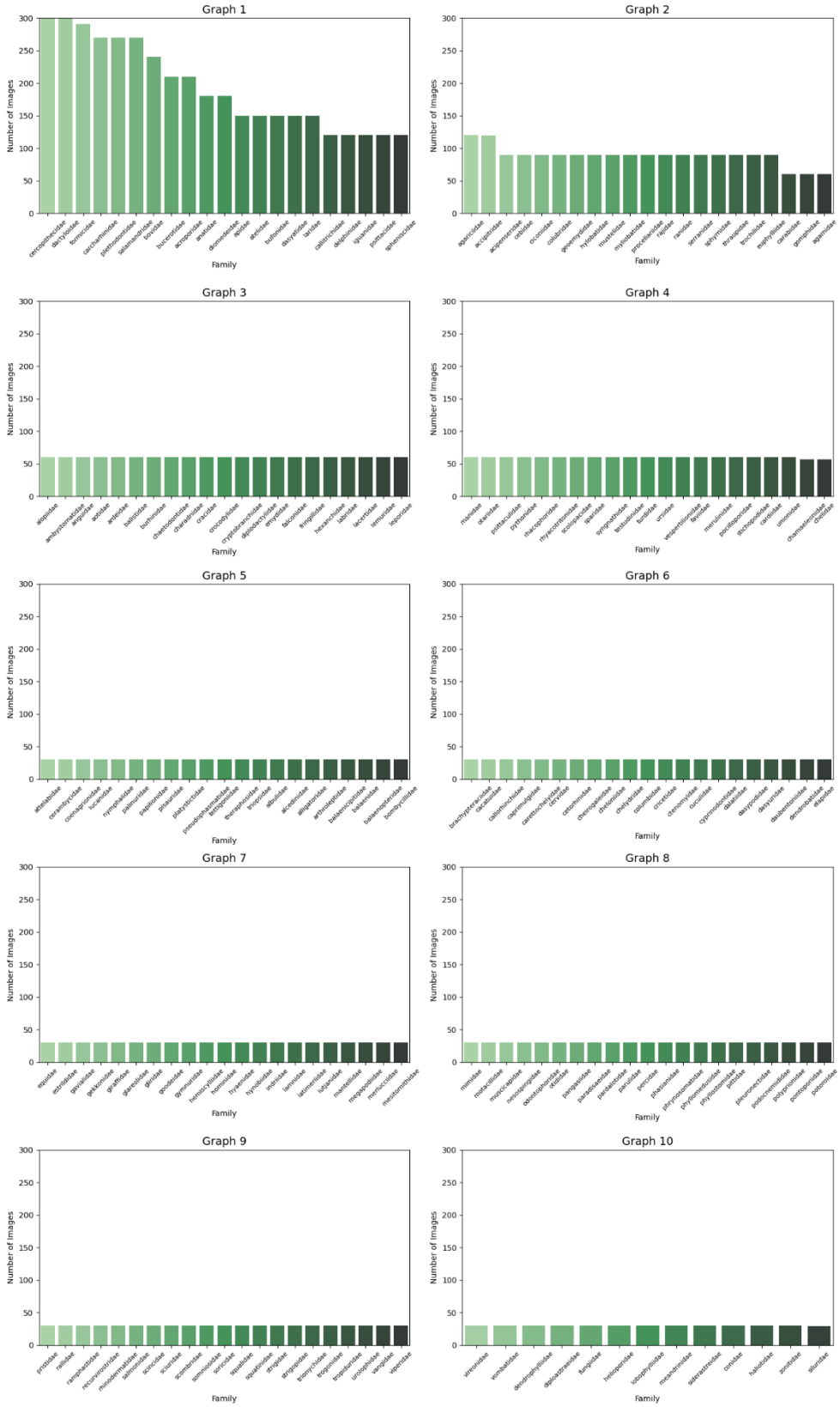

Figure 1 - Image Count Distribution per Family

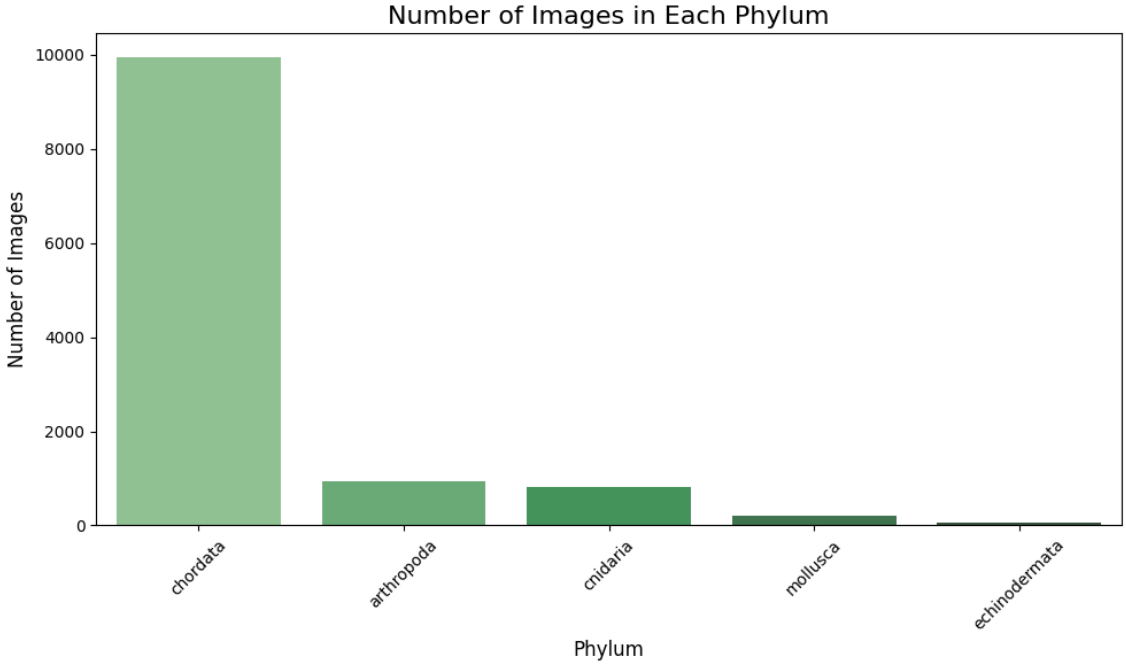

Figure 2 - Image Count Distribution per Phylum

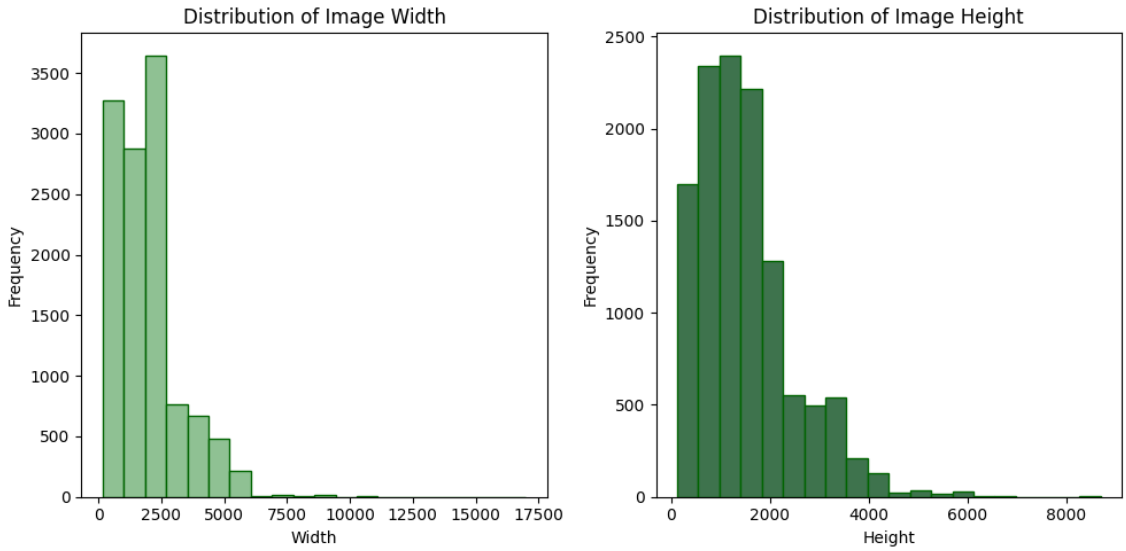

Figure 3 - Image Width and Height Distribution

Custom Model	Image Size	Batch Size	Nº of Epochs	Train Accuracy	Train F1 Score	Train Loss	Val. Accuracy	Val. F1 Score	Val. Loss
1.0	(256, 192)	16	30	0.2534	0.1849	3.2433	0.2110	0.1312	4.1051
2.0	(256, 192)	32	20	0.2058	0.1207	3.5754	0.1902	0.1014	4.0304
3.0	(256, 192)	64	33	0.2798	0.2196	3.1108	0.2219	0.1589	3.9303
4.0	(256, 256)	16	28	0.2549	0.1851	3.2492	0.2127	0.1363	4.0924
5.0	(256, 256)	32	36	0.3669	0.3240	2.5857	0.2510	0.1912	4.3325
6.0	(256, 256)	64	32	0.2821	0.2189	3.0801	0.2385	0.1749	3.9035
7.0	(384, 256)	16	9	0.1454	0.0608	3.9997	0.1393	0.0501	4.1688
8.0	(384, 256)	32	37	0.2313	0.1529	3.3693	0.1927	0.1177	3.9781
9.0	(384, 256)	64	15	0.1829	0.1059	3.7329	0.1776	0.0933	3.9850

Table 1 - Custom Models Comparison

Pre-Trained Model	Base Model	Batch Size	Nº of Epochs	Train Accuracy	Train F1 Score	Train Loss	Val. Accuracy	Val. F1 Score	Val. Loss
1.0	VGG16	16	18	0.4986	0.4425	2.4331	0.5313	0.4462	2.5536
2.0	EfficientNetB0	16	41	0.8004	0.7799	1.0873	0.7289	0.7146	1.5544
3.0	DenseNet201	16	21	0.6108	0.5675	1.9596	0.6514	0.6176	1.8593

Table 2 - Pre-Trained Models Comparison

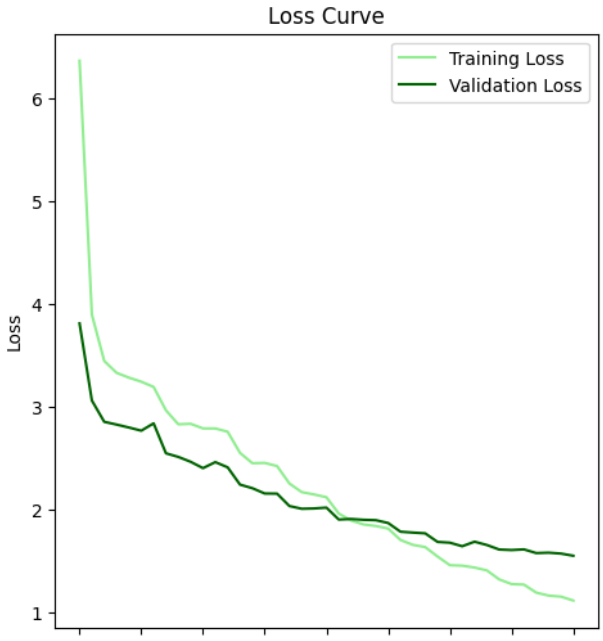

Figure 4 - Final Model's Loss Curve

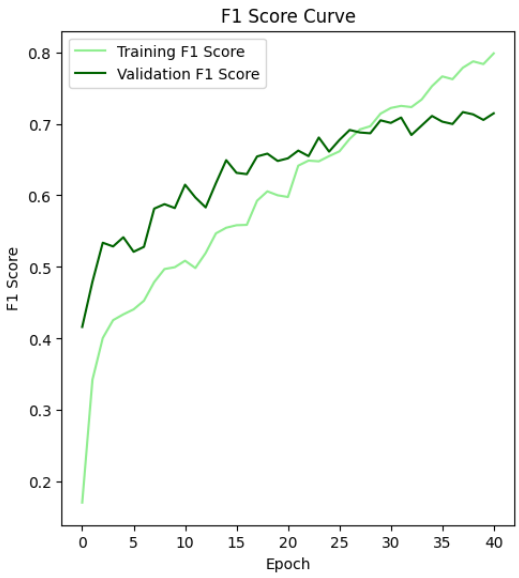

Figure 5 - Final Model's F1 Score Curve

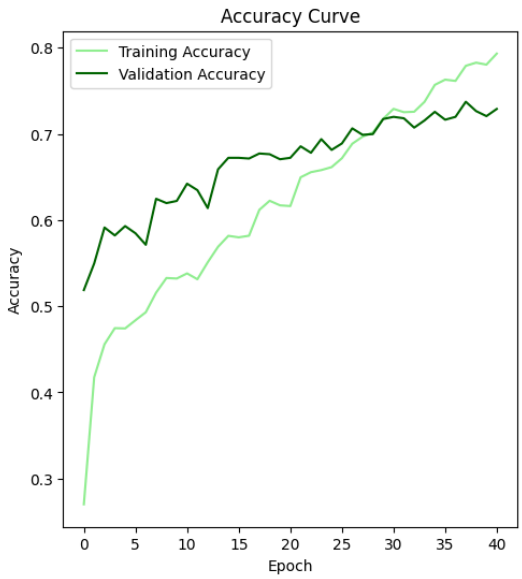

Figure 6 - Final Model's Accuracy Curve

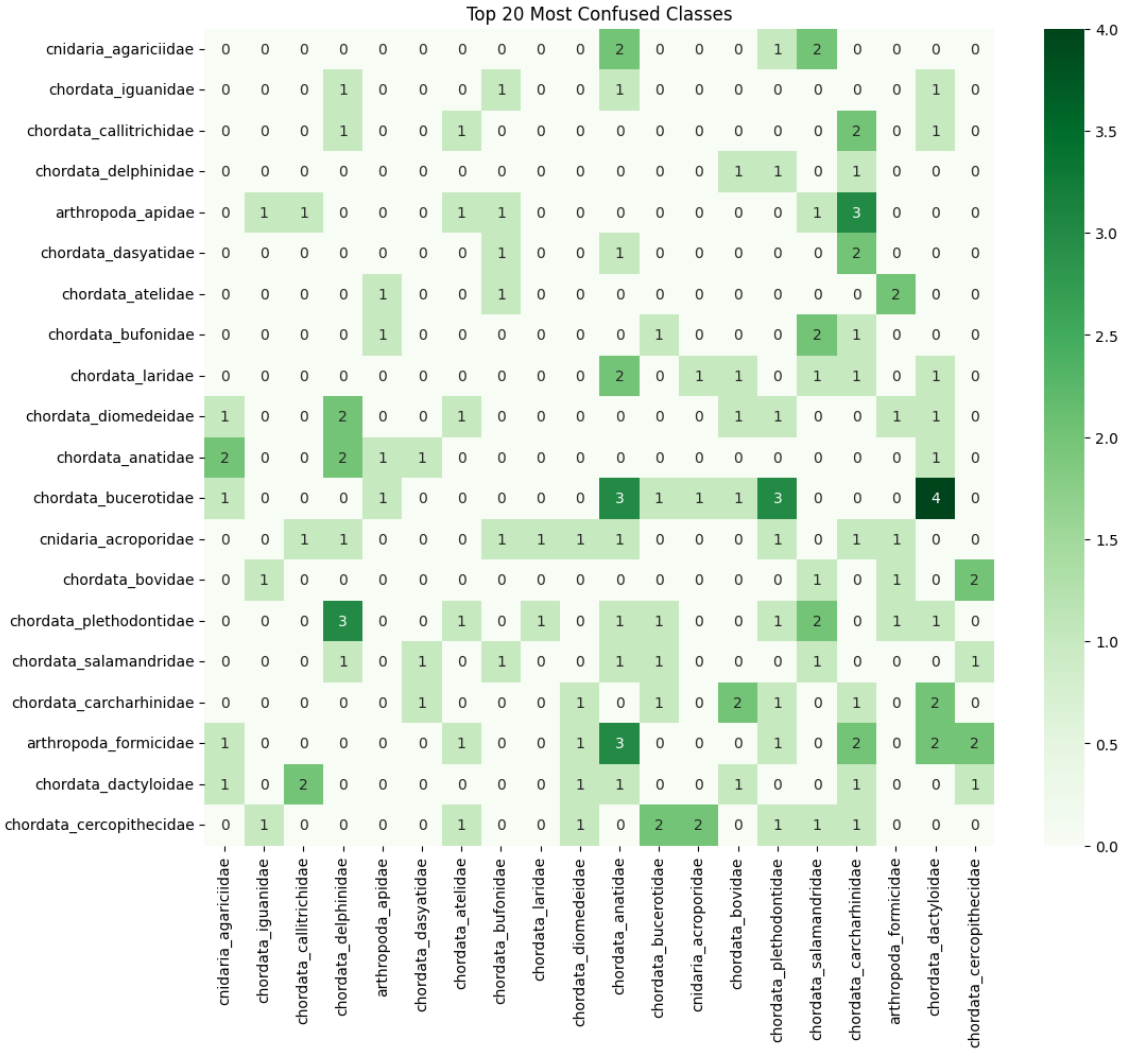

Figure 7 - Final Model's Confusion Matrix

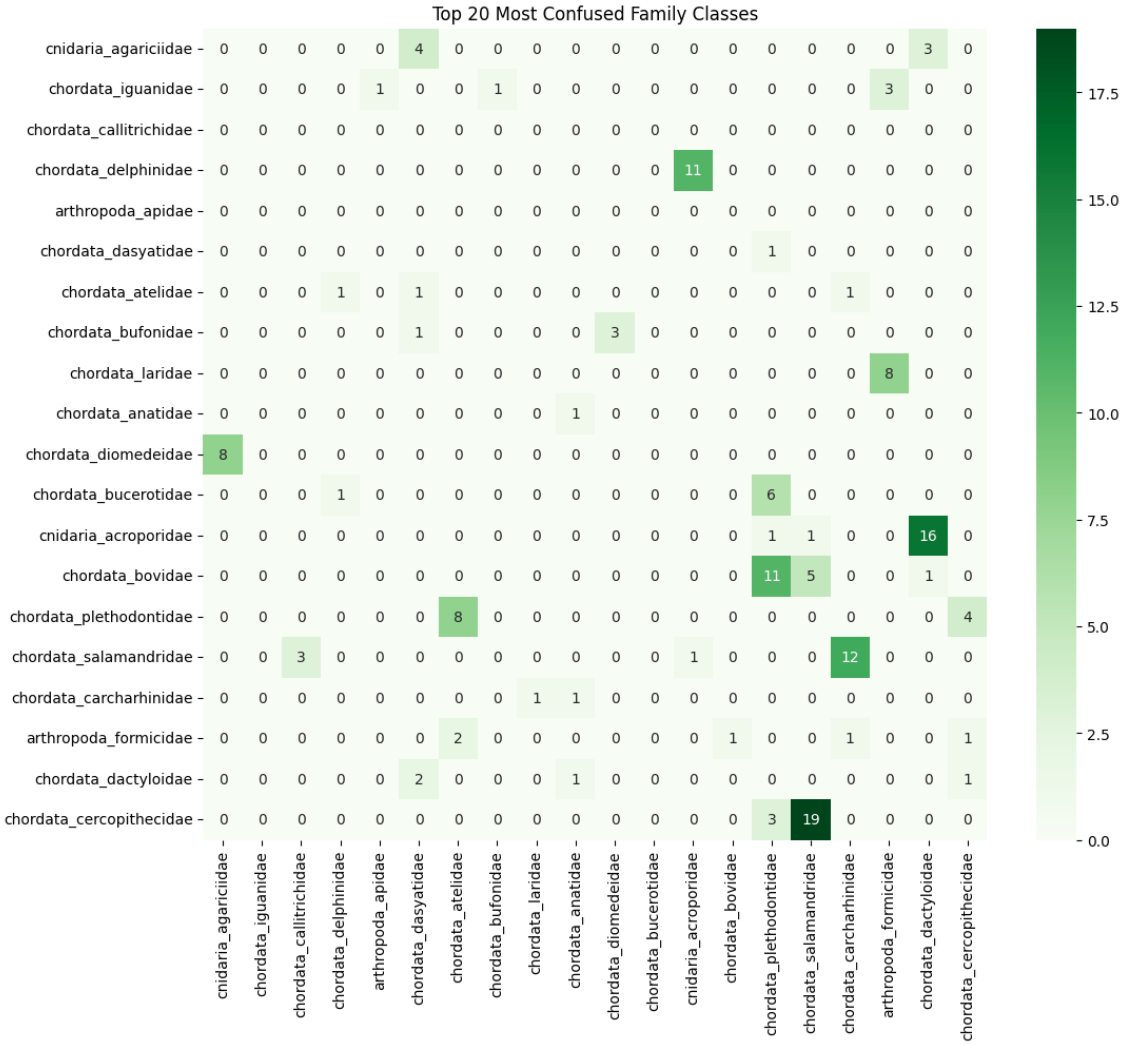

Figure 8 - Multi-level Label Model's Confusion Matrix

Test Accuracy	Test F1 Score	Test Loss
0.6904	0.7092	1.4974

Table 3 - Final Model Test Results